

---

# GPU Control 成品功能、核心逻辑与测试报告

## GPU Control – 可复现交付文档

版本：1.1.0 日期：2026-07-22 结论：代码与无 GPU 环境验证已达到“三机现场部署候选版本”；真实 Ubuntu/NVIDIA/模型/工作流验收必须按《当天部署与联调手册》执行后才能标记生产通过。

## 1. 成品范围

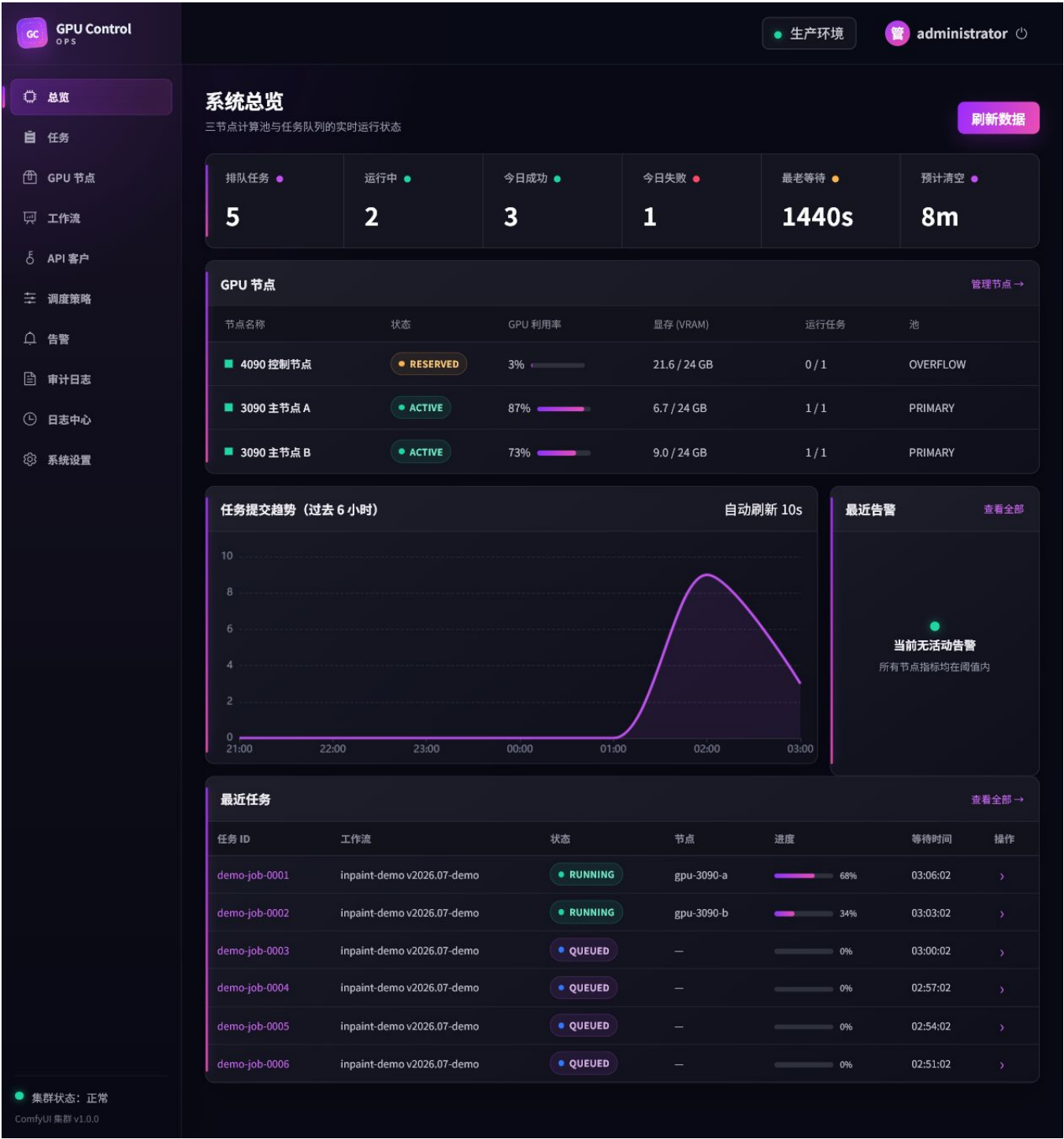
本系统不是 ComfyUI 的替代品，而是三台单 GPU ComfyUI 上方的任务、路由和运维控制层：

- PostgreSQL 持久任务队列，Redis 仅做唤醒和实时通知；
- FastAPI 公共 API、后台 API、API Key、JWT 管理登录、幂等、配额和回调；
- 单主 asyncio Scheduler，事务领取、租约、重试、恢复、取消和超时；
- 3090-A/3090-B 主池，4090 默认 RESERVED，可手工 ACTIVE 或受 Guard 控制的 OVERFLOW；
- ComfyUI HTTP/WebSocket 客户端与 Fake ComfyUI；
- Vue 3 LiClick 风格管理台；
- Dockerfile 固定构建的统一 ComfyUI 镜像，模型目录外置；
- Node Agent、gpuctl、镜像导入导出、模型同步、诊断、备份恢复；
- Prometheus、Alertmanager、Grafana、Loki、Alloy 三机日志与指标。

Celery/Flower 未采用。三张 GPU、最多三个并行槽位，不需要通用分布式任务框架；PostgreSQL 是唯一任务真相，调度与恢复语义更直接。

## 2. 主要管理功能

| 页面/接口    |  | 已实现功能                                                         |
|----------|--|---------------------------------------------------------------|
| 总览       |  | 排队、运行、今日成功/失败、最老等待、预计清空、7 小时趋势、节点、活动告警、最近任务                   |
| 任务       |  | 状态/进度/节点、管理员取消、失败重试、单任务诊断 ZIP、事件与 artifacts                   |
| 节点       |  | Drain、Reserve、Release、启动/停止<br>ComfyUI、中断、释放模型、安全重启、GPU/显存/槽位 |
| workflow |  | 导入注册包、API<br>格式验证、不可变版本、自动计算三节点兼容性、启停确认                       |
| 客户       |  | 客户配额/并发/权重、创建 API Key、Key 仅显示一次                               |
| 调度       |  | 4090 自动溢出开关、队列/等待/利用率/显存/时段阈值，数据库动态生效                         |
| 告警       |  | Alertmanager<br>鉴权入库、去重、持久投递状态、异步飞书重试、测试消息                    |
| 日志       |  | 按 job_id/request_id/node_id/error_code 构造 Grafana Loki 查询     |
| 审计       |  | 操作者、动作、目标、前后值、原因、来源 IP、request_id                             |



LiClick 风格 GPU Control 最终总览

3. 核心数据与状态机

关键表包括 jobs、job\_events、job\_attempts、node\_leases、nodes、workflows、workflow\_versions、workflow\_node\_compatibility、api\_clients、api\_keys、idempotency\_keys、job\_callbacks、alerts 和 audit\_logs。

主要状态链：

```
RECEIVED -> VALIDATING -> QUEUED -> CLAIMED -> UPLOADING
-> SUBMITTED -> RUNNING -> DOWNLOADING -> SUCCEEDED

异常出口: RETRY_WAIT -> QUEUED、FAILED、TIMED_OUT、CANCELLING -> CANCELLED
```

每次转换同时写 `job_events`；任务行保存

`request_id`、`trace_id`、`node_id`、`prompt_id`、`attempt_count`、错误码和目录。Redis 丢失不会丢任务，Scheduler 的周期扫描会重新发现 PostgreSQL 中的工作。

## 4. 事务领取与单节点单槽位

调度器只在发现空闲节点后领取一项任务，不提前向 ComfyUI 堆几十个 `prompt`。生产 PostgreSQL 的核心语义为：

```
SELECT ... FROM jobs
WHERE status = 'QUEUED'
ORDER BY effective_priority, tenant_fairness, created_at
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

领取事务中同时：

1. 锁定候选节点行；
2. 再检查 `mode`、`health`、心跳、外部队列、当前槽位；
3. 检查 workflow 兼容性和租户 `max_running`；
4. 建立唯一活动 `lease`；
5. `current_jobs + 1`；
6. 任务转 `CLAIMED` 并提交。

结束、失败、取消或恢复时释放 `lease` 并减少槽位。即使多个 Scheduler 实例误启动，单主 `advisory lock` 和行锁也阻止同一任务/槽位被重复领取。

## 5. 公平、优先级与 3090/4090 算法

候选任务按基础优先级和等待老化形成有效优先级；同优先级下选择当前未达到运行上限、最近最少被调度的租户，再按创建时间 FIFO。若第一个租户已满，查询会继续排除它寻找其他可运行租户，避免队首阻塞。

节点选择顺序：

```
可用 PRIMARY 3090 -> 最久未分配节点
否则 4090 ACTIVE -> 可立即使用
否则 4090 OVERFLOW -> 必须通过全部 Guard
否则不领取，任务继续留在 PostgreSQL
```

OVERFLOW Guard：自动开关打开；队列数量或最长等待超过阈值；4090 未人工保留；无

`sentinel`；健康在线；无外来队列；GPU 利用率不高于阈值；空闲显存不低于阈值；在允许时段内。选择节点之后、事务锁定节点之后会再次检查，减少检查与领取之间的竞争窗口。

## 6. 幂等、上传与真实工作流

同一租户和 `Idempotency-Key` 在事务中串行检查。请求摘要包含工作流、参数和上传文件 SHA-256：

- 相同 `key` + 相同摘要返回原 `job`；
- 相同 `key` + 不同摘要返回 409；
- 并发竞争由租户事务锁和数据库唯一约束收口；
- 输入先写 `.staging`，完整校验后原子移动到正式任务目录；

- 失败或竞争落败会清理 staging 和孤儿目录。

图片执行安全文件名、大小、真实解码、像素上限和 image/mask 尺寸一致性检查。真实工作流必须是 `Export Workflow (API)`，模板在入队时按白名单 bindings 注入参数，未知字段和非允许 class type 被拒绝。导入后自动生成每个节点的显存/标签兼容记录，没有兼容节点不能启用。

## 7. 维护操作协议

释放模型、停止和重启前先事务锁定节点并切为 `DRAINING`。若仍有活动 lease/任务，操作返回 409，节点保持 `DRAINING`，等待任务结束后管理员再次执行。中断会先阻止新领取、标记活动任务取消并落库，再调用 `ComfyUI /interrupt`。启动/停止/重启通过每节点独立 HMAC 密钥调用 systemd Node Agent；Agent 只能执行固定白名单命令。

## 8. 镜像一致性

统一 Dockerfile 固定：Ubuntu CUDA runtime、Python、PyTorch CUDA wheel、ComfyUI 完整 commit、自定义节点完整 commit、Python requirements。运行容器没有临时安装步骤，不使用 `docker commit`。构建后导出 `tar.gz + SHA256`，两台 3090 导入同一归档并核对 Docker image ID。

模型独立存放在三机 `/srv/comfyui/models`，挂载到容器默认 `/opt/comfyui/models:ro`。models.manifest.yaml 记录实际文件字节数和 SHA-256；空清单会直接失败，避免“没有真实模型却显示校验通过”。

## 9. 日志与诊断链路

API 中间件只生成一次 request\_id，并用于响应头、任务记录、审计和结构化日志；Scheduler 继续绑定 job\_id、node\_id、prompt\_id。三台 Alloy 采集 Docker 与 systemd journal，推送到主控 Loki；Grafana 可以用四类 ID 串联一次任务。诊断包括主机资源、GPU、Compose 状态和任务标识，敏感键按规则脱敏。

## 10. 本机真实测试结果

截至 2026-07-22 已执行：

- Ruff：通过；
- mypy strict：22 个源文件无问题；
- pytest：51 passed (34.20 秒)，包含 100 并发入队、100 个同幂等 key 只产生一个 job/目录、跨租户隔离、稳定 request\_id、告警鉴权去重、工作流兼容、动态溢出设置、节点单槽位和恢复逻辑；
- 前端 ESLint：通过；
- Prettier：通过；
- Vitest：1 passed；
- Vue TypeScript + Vite production build：通过，2038 modules transformed；
- 浏览器：管理员真实登录演示 API；总览、节点、调度页面可读；桌面无 console warning/error；390×844 无横向溢出；最终 UI 截图已保存；
- 配置：部署目录与 configs 下 12 个 YAML 文件全部解析通过；
- Alembic：空 SQLite 升级到 20260722\_0002 (head) 通过。

---

没有 Docker daemon、Linux、NVIDIA GPU、真实模型和业务 API 工作流，因此没有声称以下项目已通过：三机 Compose、systemd/UFW、NVIDIA Container Toolkit、真实推理、PostgreSQL 的真实并发锁、Loki 跨机推送、飞书和备份恢复演练。

## 11. 部署前必须提供

1. 三台固定 IP 和部署 SSH 用户；
2. 真实 ComfyUI API 工作流注册包；
3. checkpoint/VAE/LoRA/ControlNet 清单、大小和 SHA-256；
4. 自定义节点仓库、完整 commit 和 requirements lock；
5. 首单输入图、蒙版和期望业务参数；
6. 是否启用 4090 自动 OVERFLOW 及阈值；
7. 可选的飞书 Webhook/Secret、域名/正式证书。

这些外部材料不影响控制面部署，但没有真实工作流和模型就不能宣称业务出图完成。

## 12. 交付路径

- 当天执行手册：[docs/28\\_TODAY\\_DEPLOYMENT\\_MANUAL.md](#)
- 本报告：[docs/29\\_PRODUCT\\_RELEASE\\_AND\\_TEST\\_REPORT.md](#)
- 单文件 PDF：仓库根目录 GPU\_CONTROL\_成品部署联调与核心逻辑手册.pdf
- 最终界面截图：[artifacts/screenshots/gpu-control-11-final-dashboard.png](#)
- 现场状态记录：[docs/IMPLEMENTATION\\_STATUS.md](#)

# GPU Control 三机当天部署与联调手册

## GPU Control - 可复现交付文档

版本: 1.1.0 (2026-07-22) 适用: Ubuntu 24.04 LTS、RTX 4090 主控一台、RTX 3090 工作机两台 目标: 从三台可 SSH 的 Ubuntu 主机开始, 当天完成统一 ComfyUI 镜像、控制面、两台主计算节点、管理后台、日志监控和首个真实任务联调。

> 本文是实际执行顺序。示例 IP 为 192.168.10.10/11/12; 执行前替换为真实 IP。安全强化不阻塞首次上线, 但三机应位于可信局域网, 不要把 8188、9201、3100、9100、9400 暴露到公网。

## 1. 部署结果与停止条件

最终角色:

- 192.168.10.10: 4090 主控。运行 Nginx、Web、API、Scheduler、PostgreSQL、Redis、Prometheus、Alertmanager、Grafana、Loki、Alloy; 4090 ComfyUI 默认不启动, 节点状态为 RESERVED。
- 192.168.10.11: 3090-A。运行统一 ComfyUI 镜像、Alloy、主机/GPU exporter、systemd Node Agent, 节点状态为 ACTIVE。
- 192.168.10.12: 3090-B。配置同 A。
- 三台使用同一 COMFY\_IMAGE tag 和同一 Docker image ID; 模型不在镜像中, 通过 /srv/comfyui/models 同步。

遇到下列任一情况就先停止, 不继续启动业务:

- 宿主机或 CUDA 容器内 nvidia-smi 失败;
- 两台 3090 的 /system\_stats 不通;
- 三机镜像 ID 不一致或模型 manifest 校验失败;
- 真实工作流不是 ComfyUI 的 Export Workflow (API) 格式;
- API /health/ready 非 200;
- 100 并发或首个真实任务无法到达终态。

## 2. 一次性填写部署变量

在自己的变更记录中填写, 不要把密码写进聊天或 Git:

```
CONTROL_IP=192.168.10.10
WORKER_A_IP=192.168.10.11
WORKER_B_IP=192.168.10.12
LAN_CIDR=192.168.10.0/24
SSH_CIDR=192.168.10.0/24
DEPLOY_USER=ubuntu
```

```
REPO_URL='替换为仓库地址; 无 Git 时使用 scp/rsync 复制本目录'
RELEASE_REF='替换为本次交付 tag 或 commit'
COMFY_ARCHIVE=/srv/gpu-control/images/comfyui-0.1.0.tar.gz
```

端口：用户到主控 80/443；主控到工作机 8188/9201/9100/9400；工作机到主控 3100；SSH 22。PostgreSQL 和 Redis 不发布宿主端口。

## 3. 三台主机共同准备

三台分别设置唯一主机名：

```
sudo hostnamectl set-hostname gpu-control-4090 # 主控
sudo hostnamectl set-hostname gpu-worker-3090-a # A
sudo hostnamectl set-hostname gpu-worker-3090-b # B
sudo timedatectl set-timezone Asia/Shanghai
timedatectl status
ip -br address
df -h / /srv
```

三台放置完全相同的仓库：

```
sudo install -d -m 0755 /opt/gpu-control
sudo chown "$USER:$USER" /opt/gpu-control
git clone --branch "$RELEASE_REF" --depth 1 "$REPO_URL" /opt/gpu-control
cd /opt/gpu-control
chmod +x scripts/*.sh
sha256sum pyproject.toml configs/versions.lock.env deploy/*/compose.yaml
```

无 Git 时，从管理电脑把整个仓库复制到三机 `/opt/gpu-control`；不要只复制 PDF。三机上述 SHA-256 必须一致。

### 3.1 安装 NVIDIA 驱动（三台）

```
cd /opt/gpu-control
sudo scripts/install_nvidia_driver_ubuntu.sh
sudo reboot
```

重连后：

```
nvidia-smi
cat /proc/driver/nvidia/version
```

脚本使用 Ubuntu 官方 `ubuntu-drivers install --gpgpu`

自动选择计算驱动。如现场已安装可用驱动，不重复安装；直接验证 `nvidia-smi`。

### 3.2 安装 Docker、目录与 NVIDIA Container Toolkit（三台）

主控执行：

```
cd /opt/gpu-control
sudo scripts/bootstrap_control_4090.sh
sudo usermod -aG docker "$USER"
sudo scripts/install_nvidia_container_runtime.sh
```

两台 3090 各执行：

```
cd /opt/gpu-control
sudo scripts/bootstrap_gpu_node.sh
sudo usermod -aG docker "$USER"
sudo scripts/install_nvidia_container_runtime.sh
```

退出 SSH 后重新登录，再在三台验证：

```
docker version
docker compose version
```



```
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu24.04 nvidia-smi
```

容器内必须识别本机 GPU。Toolkit 固定为 1.19.1-1，安装脚本执行 `nvidia-ctl runtime configure --runtime=docker` 并重启 Docker。

## 4. 在 4090 一次生成全部配置

只在主控执行：

```
cd /opt/gpu-control
scripts/gpuctl init control \
  --control-ip 192.168.10.10 \
  --worker-a-ip 192.168.10.11 \
  --worker-b-ip 192.168.10.12
chmod 600 .env output/deploy/*.env output/deploy/INITIAL_ADMIN_PASSWORD.txt
grep -n 'CHANGE_ME' .env && echo '发现未替换占位符，停止部署' || true
```

该命令自动生成并保持一致：

- 主控 `.env`；
- 三节点清单 `configs/nodes.yaml`；
- 使用实际工作机 IP 的 `configs/prometheus.yml`；
- 两个工作机 `.env`；
- 三个互不相同的 Node Agent 密钥；
- PostgreSQL、Redis、JWT、API、Alertmanager、Grafana 密钥；
- 初始管理员密码 `output/deploy/INITIAL_ADMIN_PASSWORD.txt`。

检查：

```
grep -E '^ (CONTROL_HOST|WORKER_|COMFY_IMAGE|PUBLIC_BASE_URL)= ' .env
sed -n '1,80p' configs/nodes.yaml
grep -n '9100\|9400' configs/prometheus.yml
```

### 4.1 生成内网 TLS

```
scripts/gpuctl tls init --control-ip 192.168.10.10
openssl x509 -in deploy/control-plane/nginx/certs/server.crt -noout -subject -dates -ext
subjectAltName
```

把 `deploy/control-plane/nginx/certs/lan-ca.crt` 导入管理电脑信任库。首次命令行验收可使用 `--cacert`，不要习惯性关闭校验。

### 4.2 防火墙和主控 Node Agent

先保持第二个 SSH 会话，再执行：

```
sudo scripts/configure_ufw_control.sh \
  --lan-cidr 192.168.10.0/24 \
  --worker-a 192.168.10.11 \
  --worker-b 192.168.10.12 \
  --ssh-cidr 192.168.10.0/24
sudo scripts/install_node_agent.sh --role control
curl -fsS http://127.0.0.1:9201/health/ready | jq
```

## 5. 构建唯一 ComfyUI 镜像

构建前把真实自定义节点固定到 `docker/comfyui/custom_nodes.lock.yaml`；每个节点必须有完整 `commit` 和 `requirements lock`。不要把模型 COPY 进 `Dockerfile`，不使用 `docker commit`。

主控执行：

```
cd /opt/gpu-control
scripts/gpuctl doctor --role control
scripts/gpuctl comfy build
docker image inspect "$COMFY_IMAGE" --format '{{.Id}} {{.json.Config.Labels}}'
scripts/gpuctl image export --image "$COMFY_IMAGE" --output "$COMFY_ARCHIVE"
sha256sum --check "$COMFY_ARCHIVE.sha256"
```

镜像可能需要较长时间构建 Python、PyTorch 和 ComfyUI；看到 **构建完成** 才继续。镜像内固定 ComfyUI `commit`、自定义节点和 Python 依赖；三机模型目录统一挂载到 `/opt/comfyui/models`。

## 6. 启动 4090 控制面

```
cd /opt/gpu-control
docker compose --env-file .env -f deploy/control-plane/compose.yaml config --quiet
scripts/gpuctl deploy control
docker compose -f deploy/control-plane/compose.yaml ps
scripts/smoke_test.sh "https://192.168.10.10" \
  --ca deploy/control-plane/nginx/certs/lan-ca.crt
```

`deploy control` 会按顺序构建应用镜像、启动 PostgreSQL/Redis、升级 Alembic、写入三节点 `inventory`、幂等创建管理员、再启动完整控制面。初始管理员：

```
cat output/deploy/INITIAL_ADMIN_PASSWORD.txt
```

浏览器打开 `https://192.168.10.10`，用户名 `admin`。Grafana 位于 `https://192.168.10.10/grafana/`，密码从主控 `.env` 的 `GRAFANA_ADMIN_PASSWORD` 读取。

此时 4090 ComfyUI 不启动，属于预期；控制面和 DCGM exporter 可使用 4090，数据库中节点保持 `RESERVED`。

## 7. 配置两台 3090

主控把配置和同一个镜像发送到两台工作机：

```
scp output/deploy/worker-3090-a.env "$DEPLOY_USER@192.168.10.11:/opt/gpu-control/.env"
scp output/deploy/worker-3090-b.env "$DEPLOY_USER@192.168.10.12:/opt/gpu-control/.env"
scp "$COMFY_ARCHIVE" "$COMFY_ARCHIVE.sha256" "$DEPLOY_USER@192.168.10.11:/srv/gpu-control/images/"
scp "$COMFY_ARCHIVE" "$COMFY_ARCHIVE.sha256" "$DEPLOY_USER@192.168.10.12:/srv/gpu-control/images/"
```

3090-A 执行：

```
cd /opt/gpu-control
chmod 600 .env
sudo scripts/configure_ufw_gpu_node.sh --control-ip 192.168.10.10 --ssh-cidr 192.168.10.0/24
sudo scripts/install_node_agent.sh --role node
scripts/gpuctl image import --input /srv/gpu-control/images/comfyui-0.1.0.tar.gz
source .env
docker image inspect "$COMFY_IMAGE" --format '{{.Id}}'
```

3090-B 执行相同命令。此时只安装管理代理并导入镜像，不启动 ComfyUI；先完成第 8 节模型同步，避免服务在模型不完整时上线。

两台记录镜像 ID:

```
source .env
docker image inspect "$COMFY_IMAGE" --format '{{.Id}}'
```

结果必须与主控一致。

## 8. 模型同步与校验

把真实模型放在主控 `/srv/comfyui/models`, 按实际相对路径填写 `/srv/comfyui/models/models.manifest.yaml`。每项需要 `path`、`size_bytes`、64 位小写 SHA-256。

```
cd /opt/gpu-control
find /srv/comfyui/models -type f -not -name models.manifest.yaml -printf '%P\n'
sha256sum /srv/comfyui/models/checkpoints/真实模型.safetensors
stat -c '%s' /srv/comfyui/models/checkpoints/真实模型.safetensors
scripts/verify_models.sh
scripts/sync_models.sh --host 192.168.10.11 --user "$DEPLOY_USER" --dry-run
scripts/sync_models.sh --host 192.168.10.11 --user "$DEPLOY_USER"
scripts/sync_models.sh --host 192.168.10.12 --user "$DEPLOY_USER" --dry-run
scripts/sync_models.sh --host 192.168.10.12 --user "$DEPLOY_USER"
```

同步脚本默认不删除远端模型, 并在远端执行同一 SHA 校验。若 SSH 用户不能写 `/srv/comfyui/models`, 在两台工作机执行:

```
sudo chown -R "$USER:$USER" /srv/comfyui/models
```

然后重新同步。主控校验通过后, 在两台 3090 分别执行:

```
cd /opt/gpu-control
scripts/gpuctl models verify
GPU_CONTROL_ROLE=node scripts/gpuctl deploy node
curl -fsS http://$(hostname -I | awk '{print $1}'):8188/system_stats | jq
```

三机 `scripts/verify_models.sh` 全部显示 OK、两台工作机 `system_stats` 都返回 JSON 后才继续第 9 节联通测试。

## 9. 三机联通检查

两台工作机各执行:

```
cd /opt/gpu-control
GPU_CONTROL_ROLE=node scripts/gpuctl connectivity
systemctl is-active gpu-node-agent
docker compose -f deploy/gpu-node/compose.yaml ps
```

主控执行:

```
cd /opt/gpu-control
scripts/gpuctl connectivity --ca deploy/control-plane/nginx/certs/lan-ca.crt
curl -fsS http://192.168.10.11:8188/queue | jq
curl -fsS http://192.168.10.12:8188/queue | jq
docker compose -f deploy/control-plane/compose.yaml logs --no-color --tail 200 api scheduler
```

联通脚本检查公网入口、PostgreSQL、Redis、Loki、两台 ComfyUI、Node Agent 和 exporters。后台“GPU 节点”页应在约 20 秒内显示两台 3090 **ONLINE/ACTIVE**。

## 10. 导入真实 workflows 并提交首单

在 ComfyUI 中打开已经跑通的业务 workflow，使用 Export Workflow (API)。普通 UI 保存 JSON 不能提交。把 API JSON 放入 workflow 注册包的 `template`，同时填写：

- `workflow_key`、不可变 `version`、显示名；
- JSON Schema 参数；
- 参数到 节点号.inputs. 字段 的 bindings；
- `allowed_class_types`；
- 模型、自定义节点、最低显存、超时、输出节点。

登录管理台进入“工作流” -> “导入 workflow 包”。系统自动计算三节点兼容性；确认至少一台节点兼容后再“启用”。如果显示无兼容节点，先修模型、显存或标签，不要强行伪造兼容结果。

进入“客户”创建客户，再创建 API Key；Key 只显示一次。提交局部重绘示例（字段名固定为 `input_image` 和 `mask`）：

```
export GPU_API_KEY='gpc_复制刚创建的完整Key'
curl --cacert deploy/control-plane/nginx/certs/lan-ca.crt \
  -H "X-API-Key: $GPU_API_KEY" \
  -H "Idempotency-Key: first-production-001" \
  -F workflow_key=你的workflowkey \
  -F workflow_version=你的版本 \
  -F 'parameters={"prompt": "首单联调", "seed": 12345}' \
  -F input_image=@/path/input.png \
  -F mask=@/path/mask.png \
  https://192.168.10.10/api/v1/jobs
```

保存返回的 `job_id`：

```
JOB_ID='替换'
curl --cacert deploy/control-plane/nginx/certs/lan-ca.crt \
  -H "X-API-Key: $GPU_API_KEY" \
  "https://192.168.10.10/api/v1/jobs/$JOB_ID" | jq
curl --cacert deploy/control-plane/nginx/certs/lan-ca.crt \
  -H "X-API-Key: $GPU_API_KEY" \
  "https://192.168.10.10/api/v1/jobs/$JOB_ID/events" | jq
```

预期链路：RECEIVED -> VALIDATING -> QUEUED -> CLAIMED -> UPLOADING -> SUBMITTED -> RUNNING -> DOWNLOADING -> SUCCEEDED。结果可从 artifacts API 或管理台任务诊断包确认。

## 11. 4090 联动策略实际操作

默认只跑两台 3090。需要 4090 参与时，先在主控启动其 ComfyUI：

```
cd /opt/gpu-control
scripts/gpuctl comfy start
docker compose -f deploy/control-plane/compose.yaml --profile 4090-gpu ps comfyui-4090
```

然后在管理台“GPU 节点”对 4090：

- 点“启动服务”；
- Release 将它放入 OVERFLOW；或直接切换 ACTIVE 让三卡都可立即执行；
- 调度设置可打开 `overflow_4090_auto_enabled`，填写队列阈值、最长等待、最低空闲显存、最高 GPU 利用率、允许时段；
- Reserve 立即禁止新任务；Drain 等当前任务完成；之后可“停止服务”。

OVERFLOW 只有在两台 3090 无空闲槽且阈值满足，同时 4090 未人工保留、无

`/run/gpu-control/4090.reserved`、利用率/显存/时段均通过时才领取任务。每个 ComfyUI 始终只保留一个本系统任务。

## 12. 日志、监控和诊断

常用检查：

```
# 主控服务
docker compose -f deploy/control-plane/compose.yaml ps
docker compose -f deploy/control-plane/compose.yaml logs --since 15m api scheduler nginx

# 工作机
docker compose -f deploy/gpu-node/compose.yaml logs --since 15m comfyui alloy
sudo journalctl -u gpu-node-agent --since '15 minutes ago' --no-pager
nvidia-smi

# 集中日志
curl -fsS http://192.168.10.10:3100/ready
scripts/gpuctl diagnostics job "$JOB_ID"
```

Grafana 的 Loki 用 `job_id`、`request_id`、`node_id`、`prompt_id` 检索。管理台“日志”页面生成 Grafana 查询链接。飞书为空不影响推理；配置 `.env` 的 Webhook/Secret 后，在“告警”页发送测试消息。

## 13. 100 并发和上线验收

无 GPU 调度逻辑已由 Fake ComfyUI 测试。生产三机先用真实工作流做 1 单，再逐步 3 单、10 单，不直接并发轰炸大模型。API 入队压测：

```
python3 -m venv .load-venv
source .load-venv/bin/activate
pip install -e '[load]'
locust -f tests/load/locustfile.py --headless -u 100 -r 20 -t 2m \
--host https://192.168.10.10
```

上线签字的最小条件：

1. 三机宿主与 CUDA 容器 `nvidia-smi` 通过；
2. 三机 image ID 和模型 SHA 一致；
3. 两台 3090 为 `ONLINE/ACTIVE`，4090 为 `RESERVED`；
4. PostgreSQL/Redis/API/Nginx 就绪；
5. 首个真实任务成功且下载结果可打开；
6. 管理台 Drain/Reserve/Release/中断/释放模型/启动/停止/安全重启可操作；
7. Loki 能查到三台日志，Prometheus targets 为 UP；
8. 备份命令成功：`scripts/backup.sh`；
9. 100 并发入队无重复 job、无 5xx；
10. 未验证项如实记录，不把 Fake 测试写成真实 GPU 通过。

## 14. 快速故障定位

| 现象                | 先执行                                                                       | 常见处理                                                            |
|-------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------|
| GPU 容器启动失败        | <code>docker run --rm --gpus all ...<br/>nvidia-smi</code>                | 重装 Toolkit、执行 <code>nvidia-ctk</code> 、重启 Docker                |
| ComfyUI unhealthy | <code>docker compose ... logs comfyui</code>                              | 检查模型挂载、镜像 tag、驱动、磁盘权限                                           |
| 节点 OFFLINE        | 主控 <code>curl :8188/system_stats</code> 与 <code>:9201/health/ready</code> | 检查 IP、UFW、Agent、Compose                                         |
| 任务一直 QUEUED       | 查节点 <code>mode/health</code> 、兼容性和 <code>scheduler</code> 日志              | Release 节点、启用工作流、修 manifest                                     |
| 后台按钮 502          | <code>journalctl -u gpu-node-agent</code>                                 | 重装 Agent，检查 <code>per-node secret</code> 与 <code>sudoers</code> |
| 工作机无集中日志          | 看 worker Alloy 日志、主控 <code>:3100/ready</code>                             | 检查 <code>CONTROL_HOST</code> 与 3100 入站                          |
| 4090 不接单          | 检查是否启动、OVERFLOW 开关和全部 Guard                                               | 启动服务后 Release，或临时切 ACTIVE                                       |
| TLS 不受信任          | <code>curl --cacert ... /health/ready</code>                              | 导入 LAN CA 或换正式证书                                                |

恢复原则：先 Drain/Reserve，保留 PostgreSQL、任务目录和日志，再修服务。禁止为了排错执行 `docker compose down -v`、删除 `/srv/gpu-control/jobs` 或清空模型目录。

## 15. 当前实测边界

本交付在 Windows 无 GPU 环境已完成 Python lint、严格类型检查、51 项测试（含 100 并发和幂等竞争）、前端测试与构建。无法在本机宣称 Ubuntu、Docker daemon、NVIDIA、真实模型、真实工作流或三机网络已通过；这些必须按本文在现场执行。代码已提供全部脚本、固定镜像构建、Fake ComfyUI、检查命令和失败出口。

官方参考：

- Docker Engine Ubuntu 安装：<<https://docs.docker.com/engine/install/ubuntu/>>
- Ubuntu Server NVIDIA 驱动：<<https://documentation.ubuntu.com/server/how-to/graphics/install-nvidia-drivers/>>
- NVIDIA Container Toolkit：<<https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>>
- ComfyUI CLI  
参数：<[https://github.com/Comfy-Org/ComfyUI/blob/700821e1364eaab0e8f21c538a2131719fec57bf/comfy/cli\\_args.py](https://github.com/Comfy-Org/ComfyUI/blob/700821e1364eaab0e8f21c538a2131719fec57bf/comfy/cli_args.py)>

# 三机当天上线勾选表

## GPU Control - 可复现交付文档

本表配合 [docs/28\\_TODAY\\_DEPLOYMENT\\_MANUAL.md](#) 使用。命令和故障解释以 28 号文档为准；本表负责现场顺序、预期结果和签字记录。

### 0. 现场信息

- ☐ 日期: \_\_\_\_\_
- ☐ 操作者: \_\_\_\_\_
- ☐ 4090 主控 IP: \_\_\_\_\_
- ☐ 3090-A IP: \_\_\_\_\_
- ☐ 3090-B IP: \_\_\_\_\_
- ☐ SSH 部署用户: \_\_\_\_\_
- ☐ Ubuntu 版本三台均为 24.04 LTS: 是 / 否
- ☐ 真实 API 工作流路径: \_\_\_\_\_
- ☐ 模型清单已完成 SHA-256: 是 / 否

任一必填项为空时，不进入生产首单。

### 1. 三台空机准备

三台分别执行并记录：

```
cd /opt/gpu-control
sudo scripts/bootstrap_common_ubuntu.sh --role control # 4090
sudo scripts/bootstrap_common_ubuntu.sh --role node # 3090
sudo scripts/install_nvidia_driver_ubuntu.sh --gpgpu
sudo reboot
```

重连后安装 Toolkit，并验证：

```
sudo scripts/bootstrap_nvidia_runtime.sh
nvidia-smi
docker run --rm --gpus all nvidia/cuda:12.8.1-base-ubuntu24.04 nvidia-smi
docker version
docker compose version
```

- ☐ 4090 裸机与 CUDA 容器都识别 GPU
- ☐ 3090-A 裸机与 CUDA 容器都识别 GPU

- [ ] 3090-B 裸机与 CUDA 容器都识别 GPU
- [ ] 三台 Docker daemon 正常

## 2. 主控一次生成配置

在 4090 仓库根目录执行：

```
scripts/gpuctl init \
  --control-ip 192.168.10.10 \
  --worker-a-ip 192.168.10.11 \
  --worker-b-ip 192.168.10.12 \
  --lan-cidr 192.168.10.0/24 \
  --ssh-cidr 192.168.10.0/24 \
  --deploy-user gpuadmin
chmod 600 .env output/deploy/*.env output/deploy/INITIAL_ADMIN_PASSWORD.txt
scripts/gpuctl doctor --role control
```

- [ ] `.env` 已生成且未外发
- [ ] 两份 worker `env` 已生成
- [ ] `configs/nodes.yaml` 是三个真实 IP
- [ ] `configs/prometheus.yml` 是两个真实 worker IP
- [ ] 初始管理员密码已离线保存

## 3. 统一 ComfyUI 镜像

先补齐 `docker/comfyui/custom_nodes.lock.yaml` 的真实自定义节点完整 commit 与 requirements lock，然后在 4090 执行：

```
scripts/gpuctl image build
scripts/gpuctl image export --output /srv/gpu-control/images/comfyui-0.1.0.tar.gz
sha256sum -c /srv/gpu-control/images/comfyui-0.1.0.tar.gz.sha256
source .env
docker image inspect "$COMFY_IMAGE" --format '{{.Id}}'
```

- [ ] 构建无临时 pip/git 操作失败
- [ ] tar.gz SHA 校验通过
- [ ] 4090 image ID: \_\_\_\_\_

## 4. 启动 4090 控制面

```
sudo scripts/configure_ufw_control.sh --lan-cidr 192.168.10.0/24 --ssh-cidr 192.168.10.0/24
sudo scripts/install_node_agent.sh --role control
scripts/gpuctl tls init --control-ip 192.168.10.10
scripts/gpuctl deploy control
curl --cacert deploy/control-plane/nginx/certs/lan-ca.crt https://192.168.10.10/health/ready
docker compose -f deploy/control-plane/compose.yaml ps
```

- [ ] `/health/ready` 返回成功
- [ ] Compose 服务均为 running/healthy
- [ ] 管理后台可登录
- [ ] Grafana 可登录



- [ ] 4090 在后台显示 **RESERVED**

## 5. 两台 3090 导入同一镜像

从主控复制 worker env、镜像和 SHA 文件。两台分别执行：

```
cd /opt/gpu-control
chmod 600 .env
sudo scripts/configure_ufw_gpu_node.sh --control-ip 192.168.10.10 --ssh-cidr 192.168.10.0/24
sudo scripts/install_node_agent.sh --role node
scripts/gpuctl image import --input /srv/gpu-control/images/comfyui-0.1.0.tar.gz
source .env
docker image inspect "$COMFY_IMAGE" --format '{{.Id}}'
```

- [ ] 3090-A image ID 与 4090 相同
- [ ] 3090-B image ID 与 4090 相同
- [ ] 此阶段尚未启动 ComfyUI

## 6. 模型同步后再启动工作机

在 4090 执行 manifest 校验、dry-run 和正式同步；两台 3090 分别执行：

```
scripts/gpuctl models verify
GPU_CONTROL_ROLE=node scripts/gpuctl deploy node
curl -fsS http://$(hostname -I | awk '{print $1}'):8188/system_stats | jq
```

- [ ] 4090 模型 manifest 全部 OK
- [ ] 3090-A 模型 manifest 全部 OK, **system\_stats** 返回 JSON
- [ ] 3090-B 模型 manifest 全部 OK, **system\_stats** 返回 JSON

## 7. 三机联通与日志

```
# 两台工作机
GPU_CONTROL_ROLE=node scripts/gpuctl connectivity
systemctl is-active gpu-node-agent
docker compose -f deploy/gpu-node/compose.yaml ps

# 4090 主控
scripts/gpuctl connectivity --ca deploy/control-plane/nginx/certs/lan-ca.crt
docker compose -f deploy/control-plane/compose.yaml logs --tail 200 api scheduler
```

- [ ] 两台 3090 在 20 秒内变为 **ONLINE/ACTIVE**
- [ ] PostgreSQL、Redis、Loki、API、两台 ComfyUI 和三个 Agent 都连通
- [ ] Prometheus targets 全部 UP
- [ ] Loki 可看到三台主机日志

## 8. 导入真实工作流与首单

- [ ] 使用 ComfyUI **Export Workflow (API)**，不是普通 UI 保存 JSON
- [ ] 工作流注册包包含 **workflow\_key**、版本、API JSON、bindings、allowed class types

- [ ] 后台导入后显示三个节点兼容性
- [ ] 启用工作流后提交真实输入图和蒙版
- [ ] 状态完整经过 RECEIVED → VALIDATING → QUEUED → CLAIMED → UPLOADING → SUBMITTED → RUNNING → DOWNLOADING → SUCCEEDED
- [ ] 输出文件可下载且 SHA/尺寸校验通过
- [ ] Grafana 可用四类 ID 找到同一任务全链路

首单 job\_id: \_\_\_\_\_; prompt\_id: \_\_\_\_\_; 耗时: \_\_\_\_\_。

## 9. 管理操作实测

- [ ] Drain 3090-A: 不接新任务，当前任务结束后进入维护
- [ ] Release 3090-A: 恢复接单
- [ ] Reserve 4090: 绝不接单
- [ ] 4090 OVERFLOW: 未达到阈值不接单，达到阈值且 Guard 全通过才接单
- [ ] 中断任务
- [ ] 释放模型
- [ ] 停止、启动和安全重启 ComfyUI
- [ ] 下载诊断 ZIP

## 10. 容量与收尾

```
python3 -m venv .load-venv
source .load-venv/bin/activate
pip install -e '.[load]'
```

```
locust -f tests/load/locustfile.py --headless -u 100 -r 20 -t 2m --host https://192.168.10.10
```

- [ ] Fake ComfyUI 100 并发无丢单和重复领取
- [ ] 真实工作流完成 1、3、10 单阶梯验证
- [ ] 备份脚本成功并记录文件 SHA
- [ ] 4090 恢复 RESERVED
- [ ] 两台 3090 保持 ACTIVE
- [ ] 现场未通过项已写入 IMPLEMENTATION\_STATUS.md

现场结论: 通过 / 有条件通过 / 不通过。操作者签字: \_\_\_\_\_; 复核人: \_\_\_\_\_。